COMPUTER SCIENCE + COMPUTER ENGINEERING

Computer Systems and Software Engineering

Student handbook / Expanded Volume 04

From native instructions to operating systems, bounded concurrency, local protocols and defensible engineering.

20 expanded lessons · 100 lesson questions · 20 runnable examples

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Contents

Module 16: Processor architecture and assembly 4

16.1 — Instruction sets, registers and calling conventions 4

16.2 — Pipelines, dependencies and branch prediction 7

16.3 — Caches, locality and memory hierarchy 9

16.4 — Virtual memory, SIMD and architectural limits 10

Module 17: Operating systems and resource management 14

17.1 — Processes, threads, system calls and protection 14

17.2 — Allocation, paging and memory-mapped data 16

17.3 — Scheduling, responsiveness and real-time constraints 17

17.4 — Filesystems, persistence and asynchronous I/O 19

Module 18: Concurrency and parallel programming 23

18.1 — Races, mutexes and happens-before 23

18.2 — Condition variables, deadlocks and cancellation 25

18.3 — Atomics, memory ordering and false sharing 27

18.4 — Task graphs, work stealing and scaling limits 29

Module 19: Networks, storage and data services 32

19.1 — Network layers, latency and transport semantics 32

19.2 — Protocol design, serialization and timeouts 34

19.3 — Relational modeling, SQL and indexes 35

19.4 — Transactions, consistency and distributed failure 37

Module 20: Engineering practice, security and maintainability 41

20.1 — Requirements, architecture and design records 41

20.2 — Defensive programming and threat modeling 43

20.3 — Build automation, dependencies and release discipline 45

20.4 — Ethics, accessibility, teamwork and sustainable delivery 47

Sources and reading directory 50

Expanded Volume 04 · Package 05

This volume connects native game code to processor execution, operating systems, concurrency, network protocols, persistent data and engineering practice. Its output is a small, defensible systems application—not a graphical engine yet.

Sequence: Modules 16–20, lessons 16.1–20.4, Gate D. The original module IDs, titles, outcomes, practical briefs and three-question quizzes are retained. The expanded teaching sections, worked implementations, two added questions per lesson, problem sets and ForgeQueue project realization are new instructional material. The untouched source slice is in `source/original_volume_04.json`.

Prerequisites: Demonstrated Volume 3 work in C++ ownership, containers, algorithms and language implementation. Start with `START_NEXT.pdf` and the twelve readiness questions. The source lists “Volumes 01–04” as Module 20 prerequisites; operationally, complete the previous foundations and Modules 16–19 before integrating Module 20. This clarification does not require Module 20 to precede itself.

Study allowance: The source assigns 380–600 planning hours to this volume, including assigned reading, exercises, experiments, integration and review. Reading this handbook alone does not meet that workload or demonstrate competence.

How to use each lesson

Read the original foundation, work through the expansion and predict the example before running it. Open the named implementation when the example calls a helper. Complete the original lab and the expanded evidence steps in the workbook; a small function check is not a substitute for the broader laboratory. Answer the five questions unaided, then use the separate answer key. Solve the transfer task independently and preserve the evidence.

Each question scores 0–2. Suggested formative pass is 9/10, including at least 5/6 on the retained original questions, plus the reading, lab and transfer evidence. The first three questions are retained; questions four and five are additions. Passing the supplied reference tests does not complete student work.

Five evidence categories

A **contract** states required behavior. A **proof or argument** establishes a property under assumptions. A **model** predicts behavior in a simplified system. A **test** observes selected executions. A **measurement** describes a declared workload and environment. Keep these categories separate: neither a benchmark nor a sanitizer result proves all behavior.

C++20 is the native teaching baseline. Python provides portable orchestration, protocol and SQLite experiments. Native C++ threads and atomics are used for C++ memory-model lessons; Python tests are not a replacement for those semantics. All network examples are local-only. The selected native worker is trusted course code, not arbitrary uploaded code.

Reference directory

R-codes identify inherited curriculum readings; P-codes identify official documentation added for this expansion. Read the sections assigned by each lesson. The full clickable directory is in `SOURCES.md` and at the end of this handbook. External books, specifications and courses are linked, not copied in full. Tool documentation should match the installed version.

Module 16: Processor architecture and assembly

Original prerequisites: Modules 11-15.

Integration goal: Machine-level investigation. Choose a bounded entity-data loop that produces a checked result. Explain its host assembly and compare it with the selected RISC-V teaching routine without treating the two targets as identical.

16.1 — Instruction sets, registers and calling conventions

Learning objective: Read a small machine-level routine without confusing ISA and implementation.

Original foundation

An instruction-set architecture defines visible operations, registers and relevant behavior. A microarchitecture implements that contract using particular pipelines, caches and execution units. Different implementations can run the same machine code while achieving different performance. A calling convention specifies how functions pass arguments, preserve registers and return values. Begin with a small load/store ISA such as the selected RISC-V subset, then compare your host's generated assembly. Do not assume an educational subset implements every extension or privileged feature. Assembly understanding helps explain costs and interoperability, but handwritten assembly is rarely the first optimization step.

Expanded study

Three contracts, not one

A compiler translates language rules into instructions; the instruction-set architecture defines what those instructions mean; the application binary interface defines how separately compiled routines cooperate. A correct arithmetic routine can still be unusable if it destroys a register the caller expects to survive. Conversely, a routine that obeys its calling convention may still compute the wrong sum. Keep a separate checklist for arithmetic, memory validity and call-boundary behavior. [R15, P01, P16]

Read the frame before reading the loop

The supplied RV32I routine accepts a pointer in a0 and a count in a1 and returns the low 32 bits of the sum in a0. It subtracts 16 from sp, saves ra at offset 12 and s0 at offset 8, then uses s0 as its accumulator. Those locations are relative to the adjusted stack pointer. On return it restores both registers and restores sp. Saving ra is redundant for this leaf function; it is included to make the frame exercise explicit, not as a performance recommendation. [P01, P16]

Prove the memory range

Let the original pointer be p and count be n. Before iteration k, the current pointer is p + 4k, the remaining count is n-k, and the accumulator contains the sum of the first k words modulo 2^32. The branch before the loop handles n=0 without a load. The caller must still provide readable aligned storage for all n words; the assembly routine cannot infer the true allocation length from a pointer. The C++ span expresses a view length, but its caller must also create a valid view.

Compare targets without pretending to emulate them

Run the host C++ example to check the arithmetic and inspect native/kernels.cpp in generated host assembly. Annotate native/asm/sum_u32_rv32.S separately. The C++ demonstration is not an instruction emulator, and assembling the RISC-V file is not the same as executing it on a RISC-V processor. Instruction spelling and register names will differ on x86-64 and Arm. Record the target triple and ABI whenever you save an assembly listing. [P02, P03]

Worked reasoning

For input words [7,11,0,5], the accumulator after each load/add is 7,18,18,23. The pointer advances by 16 bytes in total and the count reaches zero. With [4294967295,2], the result is 1 modulo 2^32. That machine-level wrapping contract is modeled with an unsigned C++ type; it is not permission to overflow a signed C++ int. Inspect all fifteen instruction lines in the supplied assembly, including the restore path, not just the five loop-body instructions.

Runnable example

File: `native/examples/lesson_16_1.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.1 --build-dir build
```

```
#include "lab_16_1.hpp"
#include <array>
#include <iostream>
int main(){std::array<std::uint32_t,4>a{7,11,0,5};std::cout<<"sum="<<course::sum_u32(a)<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
sum=23
```

Assembly annotation exercise

This is a separate RV32I teaching routine. The host demonstration does not emulate these instructions. Label the frame, caller preconditions, memory operations and restore path.

```
# Hand-authored RV32I teaching routine, ILP32 integer calling convention.
# Input a0: aligned pointer to n readable uint32 words; a1: n.
# Output a0: sum modulo 2^32. Caller provides valid memory and stack.
# ra save is deliberately redundant for this leaf; useful for frame annotation.
.text
.globl course_sum_u32
course_sum_u32:
    addi sp, sp, -16
    sw ra, 12(sp)
    sw s0, 8(sp)
    addi s0, zero, 0
    beq a1, zero, .Ldone
.Lloop:
    lw t0, 0(a0)
    add s0, s0, t0
    addi a0, a0, 4
    addi a1, a1, -1
    bne a1, zero, .Lloop
.Ldone:
    addi a0, s0, 0
    lw s0, 8(sp)
    lw ra, 12(sp)
    addi sp, sp, 16
    jalr zero, 0(ra)
```

Practical checkpoint

Annotate a ten-to-twenty-instruction routine with register meanings and memory accesses. Pass: predict its output for two inputs and identify caller/callee responsibilities from the chosen ABI reference.

Expanded contract: Native sum_u32(span<const uint32_t>) returns a modulo-2^32 sum; empty input returns zero and input is unchanged. The span must refer to valid live objects. No RISC-V execution is claimed by this helper.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_1.hpp`. Use `python3 check_lab.py 16.1 --student` after implementation.

Independent transfer

A caller keeps an important value in s0. Your sum routine returns 23 but leaves s0 equal to 23. Is the call correct?

Quiz — answer before opening the key

1. **How does an ISA differ from a microarchitecture?**
2. **What does a calling convention specify?**
3. **Why identify the exact ISA subset in a simulator?**
4. **What stack address contains the saved s0 after sp has been decremented by 16?**
5. **What result should the unsigned 32-bit reference produce for [4294967295,2], and why not use signed overflow?**

Assigned study

R15: Assembly Language and Procedures and Stacks; R35: selected base integer ISA.

Expansion source map: R15, R35, P01, P02, P03, P16. See the source directory for the exact documents and version notes.

16.2 — Pipelines, dependencies and branch prediction

Learning objective: Explain why instruction count alone does not predict latency.

Original foundation

A pipeline overlaps stages of multiple instructions. Dependencies can prevent later work from proceeding until earlier results are ready. Modern processors may issue or complete operations out of order while preserving required architectural behavior. Branch prediction speculates about control flow, and mispredictions can discard work. Throughput measures sustained work per time; latency measures delay for a particular result. A long dependency chain can have poor latency even when the processor can execute many independent operations per cycle. Do not infer performance from clock frequency alone. Instruction mix, memory stalls and available parallelism matter.

Expanded study

Build a dependency graph

Each addition to one accumulator consumes the previous accumulator value. Even if a processor can start multiple arithmetic instructions in one cycle, it cannot use a result that is not ready. Draw arrows between dependent operations before counting instructions. Loads may supply independent work, but their addresses, miss behavior and use dependencies can introduce other constraints. A branch adds control dependence, which prediction attempts to hide speculatively. [R15]

Use lower bounds honestly

In an invented machine model with four-cycle add latency, eight dependent additions have a 32-cycle dependency bound. With four independent two-add chains, the longest arithmetic chain is eight cycles. If the machine can issue one such addition per cycle, eight instructions also require at least eight issue cycles. These are lower bounds, not an exact prediction: starting the chains, loading operands, merging partial sums and recovering from branches still cost work.

Preserve semantics when exposing parallelism

The reference uses four unsigned 64-bit accumulators and combines them after the loop. Unsigned modular addition allows this regrouping. A floating-point sum needs a different contract because changing grouping can change rounding. Do not label a faster, numerically different result an improvement without deciding whether the difference is acceptable. The final tail also matters: seven elements include one four-element block and three remaining elements.

Measure the implementation that survived compilation

The native benchmark places kernels in a separate translation unit, produces a checked checksum, performs warmup and changes variant order. Those measures make the experiment interpretable, but assembly inspection is still required: an optimizing compiler might vectorize both variants or otherwise erase the difference you intended to test. A passing checksum establishes agreement on tested data, not a universal statement about every processor. Preserve raw trials instead of reporting only the fastest run. [P02, P03]

Worked reasoning

For [1,2,3,4,5,6,7], the first block gives accumulators (1,2,3,4). The tail adds 5,6,7 to the first accumulator, producing (19,2,3,4), whose sum is 28. This tail placement is deliberately simple; it does not mean the compiler issues the instructions in that order. Run benchmark in a Release build, save its CSV, and explain whether generated code still distinguishes the two source strategies.

Runnable example

File: `native/examples/lesson_16_2.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.2 --build-dir build
```

```
#include "lab_16_2.hpp"
#include <array>
#include <iostream>
int main()
{std::array<std::uint64_t,7>a{1,2,3,4,5,6,7};std::cout<<"sum="<<course::sum_split(a)<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
sum=28
```

Practical checkpoint

Compare dependent and independent arithmetic loops while preventing dead-code elimination. Pass: preserve observable results, inspect generated code and explain measured differences as evidence rather than universal CPU facts.

Expanded contract: Native sum_split(span<const uint64_t>) returns the modulo-2^64 sum without modifying input. All tail lengths are supported. Performance is an experiment, not a required speedup.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_2.hpp`. Use `python3 check_lab.py 16.2 --student` after implementation.

Independent transfer

Four accumulators are slower on a 12-element list. What should be concluded?

Quiz — answer before opening the key

1. **What is a data dependency?**
2. **Does higher clock frequency guarantee a faster program?**
3. **Why can extra independent accumulators help?**
4. **A model says add latency is four cycles and issue rate is two adds per cycle. What bounds apply to eight dependent adds?**
5. **Why is a checksum necessary but not sufficient for a benchmark?**

Assigned study

R15: Pipelining and Performance Measures.

Expansion source map: R15, R11, P02, P03. See the source directory for the exact documents and version notes.

16.3 — Caches, locality and memory hierarchy

Learning objective: Relate access patterns to cache lines and working sets.

Original foundation

Processors use a hierarchy of storage with different capacities and access costs. Caches transfer data in blocks called cache lines. Spatial locality reuses nearby data; temporal locality reuses recently accessed data. A working set is the data actively needed during a phase. Once it exceeds a cache level, behavior can change sharply. Cache associativity and replacement influence conflicts. A cache miss is not always a main-memory access, because another cache level may satisfy it. Prefetching can help predictable patterns but can also waste bandwidth. Measure several working-set sizes and avoid attributing every slow access to one guessed cache characteristic.

Expanded study

Translate addresses into model state

A direct-mapped cache model needs an explicit line size, number of lines, initial validity and replacement behavior. For byte address A, line size B and line count L, the memory block is floor(A/B), the cache index is block mod L, and the tag is floor(block/L). A hit requires both a valid line and a matching tag. Index equality alone is not a hit: different blocks compete for the same slot.

Separate locality from residency

Temporal locality reuses a recent value; spatial locality accesses nearby values that may share a fetched line. Both are properties of an access stream relative to a storage hierarchy. A large sequential pass can have good spatial locality even when little data is reused. Repeatedly revisiting a small set can have good temporal locality. Neither property guarantees a hit if competing accesses evict the data. [R15]

Avoid misleading stride comparisons

Reading every sixteenth element performs less useful work than reading every element. Comparing their elapsed times alone is not a fair locality experiment. The supplied locality benchmark instead visits all residue classes for a stride of sixteen so every element is visited exactly once. All three patterns use index arrays and check the same sum. Index-array traffic is included in the timed loop; this is a comparison of those complete access paths, not a pure measurement of DRAM latency.

Keep model and hardware reports separate

The teaching cache has two lines of sixteen bytes in the worked example. These are chosen model parameters, not information about the learner's machine. The host has additional cache levels, prefetchers, translation effects and shared-resource interference. A timing step can suggest a working-set boundary but cannot identify its cause uniquely. Record data bytes, index bytes, build settings and trial order. Do not use a tiny model to claim a precise cache-miss count for the real program. [R15, P02, P03]

Worked reasoning

With L=2 and B=16, address 0 maps to index 0/tag 0 and misses. Address 4 shares that line and hits. Address 32 maps to index 0/tag 1 and replaces it. Address 0 therefore misses again. Addresses 16 and 20 map to index 1/tag 0 and produce a miss then a hit. The complete sequence is M,H,M,M,M,H: two hits in six accesses. The model does not include dirty writeback, coherence or prefetching.

Runnable example

File: `native/examples/lesson_16_3.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.3 --build-dir build
```

```
#include "lab_16_3.hpp"
#include <iostream>
int main(){course::DirectCache c(2,16); for(std::uint64_t a:
{0,4,32,0,16,20})std::cout<<a<<':'<<(c.access(a)?"hit":"miss")<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
0:miss
4:hit
32:miss
0:miss
16:miss
20:hit
```

Practical checkpoint

Benchmark sequential, strided and randomized access over multiple array sizes. Pass: record assumed versus measured hardware information and explain spatial locality without claiming exact cache behavior from one timing curve.

Expanded contract: DirectCache(lines,line_bytes): lines in 1..65536, line_bytes in 1..1048576. access(uint64 address) returns a model hit and updates the selected line; reset invalidates all lines. Every modeled access fits in one line.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_3.hpp`. Use `python3 check_lab.py 16.3 --student` after implementation.

Independent transfer

Your timing curve slows at a particular array size. Does that prove the L2 size equals the array size?

Quiz — answer before opening the key

1. **What is spatial locality?**
2. **How many 4-byte values fit in a hypothetical 64-byte line?**
3. **Does every first-level cache miss access DRAM?**
4. **In the two-line, sixteen-byte model, why do addresses 0 and 32 conflict?**
5. **Why does the benchmark traverse every stride residue?**

Assigned study

R15: Caches and the Memory Hierarchy.

Expansion source map: R15, P02, P03. See the source directory for the exact documents and version notes.

16.4 — Virtual memory, SIMD and architectural limits

Learning objective: Recognize address translation and vector execution as separate mechanisms.

Original foundation

Virtual addresses belong to a process view; address translation maps them to physical storage subject to permissions. A translation lookaside buffer caches recent translations. Large working sets and access patterns can stress translation independently of ordinary data caches. Single-instruction multiple-data execution applies operations across several lanes. It benefits suitable regular work but needs attention to alignment, tails, masks and numerical semantics. Vector width is not a guaranteed speedup factor. Some workloads are limited by memory bandwidth or serial dependencies instead. Later co-design lessons will connect these observations to roofline and Amdahl-style limits.

Expanded study

Two different caches answer different questions

A data cache supplies recently used contents; a translation lookaside buffer supplies recent virtual-address translation information. A TLB miss is not automatically a missing data page. A page-table walk may find a present, permitted mapping, after which the data access can still hit or miss ordinary caches. A page fault can also be about permissions rather than disk I/O. Keep the address-translation path separate from the data path in a diagram. [R15, R16]

A lane diagram is not a speedup guarantee

A vector instruction can apply a suitable operation to several lanes, but the surrounding algorithm still loads operands, handles tails and stores results. Bandwidth, dependencies and instruction availability can limit the gain. The teaching add_blocks helper uses blocks of four scalar operations; it is not written with target-specific vector intrinsics. Only generated code or a documented target implementation establishes whether SIMD instructions are actually used. [P02, P03]

Alignment is a contract, not a cast

A span starting at the second uint32_t of a correctly allocated array is still properly aligned for uint32_t. It may no longer begin on a preferred vector boundary. That is a safe way to test an offset view. Casting an arbitrary byte address to a wider aligned type is not an equivalent experiment and can violate alignment or object-lifetime rules. Preserve typed storage and use legal accesses while studying compiler behavior.

Bound the last block and document aliasing

For n values and a block width W, complete blocks cover floor(n/W)*W values. The remainder must be processed without reading past the input or writing past the output. The helper permits disjoint output or exact whole-span aliasing with an input. Partial overlap is excluded: writing out[i] could overwrite a later input. A faster vector loop that silently relies on nonoverlap needs an explicit precondition, not an optimistic comment.

Worked reasoning

For virtual address 0x12345 and a chosen 4096-byte page size, the virtual page number is 0x12 and the page offset is 0x345. This arithmetic does not reveal a physical frame number. For five additions at width four, one complete block handles indices 0..3 and the tail handles index 4. The demo computes [11,22,33,44,55]. Sentinel elements around an offset output view must remain unchanged.

Runnable example

File: `native/examples/lesson_16_4.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.4 --build-dir build
```

```
#include "lab_16_4.hpp"
#include <array>
#include <iostream>
int main()
{std::array<std::uint32_t,5>a{1,2,3,4,5},b{10,20,30,40,50},o{};course::add_blocks(a,b,o);for(auto
v:o)std::cout<<v<<' ';std::cout<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
11 22 33 44 55
```

Practical checkpoint

Inspect compiler vectorization reports for a simple loop and compare aligned/misaligned or tail cases safely. Pass: test lengths 0, 1, lane_width-1 and lane_width+1 and explain translation versus caching.

Expanded contract: Native add_blocks(a,b,out) requires equal lengths and valid live spans; uint32_t addition wraps. Disjoint output or exact full alias is supported; partial overlap is outside the contract. Mismatched lengths throw before modification.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_4.hpp`. Use `python3 check_lab.py 16.4 --student` after implementation.

Independent transfer

The compiler refuses to vectorize an overlapping-buffer loop. Should the programmer simply assert nonaliasing?

Quiz — answer before opening the key

1. **What does a TLB cache?**
2. **Does four-lane SIMD guarantee a fourfold speedup?**
3. **Why test array lengths not divisible by vector width?**
4. **How many complete width-eight blocks and remaining elements occur for n=19?**
5. **Why can a TLB miss happen without disk access?**

Assigned study

R15: Virtual Memory and Parallel Processing; R11: bounds and arithmetic contracts.

Expansion source map: R15, R16, R11, P02, P03. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Trace a compiled C++ loop through a small assembly example and a cache model. Deliver an instruction/data-flow explanation and a controlled locality experiment.

Choose a bounded entity-data loop that produces a checked result. Explain its host assembly and compare it with the selected RISC-V teaching routine without treating the two targets as identical.

1. A register/memory trace for two inputs and the zero-length case; exact target and ABI.
2. A cache-model worksheet with stated line/entry assumptions and at least one conflict trace.
3. An equal-useful-work sequential/strided/random locality experiment at three working-set sizes.
4. A vectorization report and tail tests; explain whether observed assembly actually uses SIMD.

5. A report separating instruction semantics, analytical cost models and measured timing.

Live review: Change the line size in the model, then change the array length to one beyond a vector block. Predict the changed evidence before rerunning.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.

Module 17: Operating systems and resource management

Original prerequisites: Module 16.

Integration goal: Operating-system investigation. Use only ordinary user privileges and disposable files. Compare execution and storage mechanisms while keeping the contents and useful work constant.

17.1 — Processes, threads, system calls and protection

Learning objective: Explain which resources are isolated and which are shared.

Original foundation

A process is an execution environment with an address space and operating-system-managed resources. Threads within a process share memory but have their own execution state. A system call requests a kernel service through a controlled boundary. User/kernel separation protects privileged operations; it does not make application code automatically safe. Process isolation helps contain faults, while threads allow lower-overhead sharing at the cost of synchronization complexity. Starting a worker process, a thread and an asynchronous task are not interchangeable choices. Identify ownership, failure propagation and communication needs before choosing a mechanism.

Expanded study

Make an ownership map first

A thread has its own execution state but can reach the process's shared objects. A separate process normally needs an explicit exchange to transfer information. Neither choice automatically answers who owns an input buffer, who reports an exception, or who waits for completion. Draw the worker's input, output and lifetime before selecting the mechanism. A future is a handle to a result, not a new protection boundary by itself. [R16, P08, P14]

Use a process boundary for a narrow experiment

The reference launches a fixed Python child program with an argument list and no shell. The parent validates at most 256 small integers, serializes them to standard input, and receives a result plus the child's process ID. The child has a fresh interpreter and reconstructs its own list from bytes. Changing that list would not directly mutate the parent's original list. Shared-memory APIs could change the design, but are not used here. [P08]

Failure belongs in the interface

The parent sets a timeout, checks the exit status and parses the response. A nonzero exit is not a valid numerical result; malformed output is not success just because the process ended. A timeout requires cleanup and a clear error path. If a worker can perform durable external effects, termination alone cannot undo them. The fixed child in this lab only computes and prints, making the boundary small enough to reason about. [P08]

Do not mistake isolation for a sandbox

A child typically starts with access derived from the parent account. It can still read or write whatever its environment permits unless additional controls are imposed. The exercises never launch untrusted source code or request-supplied executable paths. Use a private scratch directory and ordinary user permissions. Operating-system isolation is useful fault containment, but the teaching subprocess wrapper is not a security boundary for hostile tools.

Worked reasoning

The parent sends [2,3,5]. The child computes 10 and reports a process ID different from the parent's. The supplied demo deliberately prints only whether the identities differ, because actual numeric IDs vary between runs. Compare that with a C++ thread capturing a reference to one Inventory object: the thread can reach the same object, so its updates require the locking contract introduced in Module 18.

Runnable example

File: `python/examples/lesson_17_1.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.1 --build-dir build
```

```
import os
from python.reference.lab_17_1 import run_child
r=run_child([2,3,5])
print("sum=",r["sum"]," separate_process=",r["pid"] != os.getpid(),sep="")
```

Expected output from the supplied reference in the validated environment:

```
sum=10 separate_process=True
```

Practical checkpoint

Write a program that reports its process identity and runs independent worker tasks. Pass: explain what memory is shared in the chosen approach and how worker failure is reported without privileged operations.

Expanded contract: Python run_child(list[int],timeout=3.0) accepts at most 256 integers in 0..1000 and timeout in (0,30]. It runs only a fixed child program, returns integer pid/sum fields, and preserves the input. Errors propagate; no shell is invoked.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_1.py`. Use `python3 check_lab.py 17.1 --student` after implementation.

Independent transfer

A child process can still overwrite a file owned by the same account. Does that contradict process memory isolation?

Quiz — answer before opening the key

1. **What do threads in one process typically share?**
2. **What is a system call?**
3. **Why might processes be preferable to threads for a risky tool?**
4. **Why is an argument list with shell=False useful here?**
5. **What is wrong with returning zero whenever a worker crashes?**

Assigned study

R16: Processes, Process API and Threads.

Expansion source map: R16, P08, P14. See the source directory for the exact documents and version notes.

17.2 — Allocation, paging and memory-mapped data

Learning objective: Distinguish reserved address space, committed storage and live objects.

Original foundation

An allocator manages storage for application objects; the operating system manages virtual memory and backing resources. Reserving an address range is not the same as having every page physically resident. Page faults can occur when access requires additional mapping or data movement. Application allocation behavior and operating-system paging interact but are not identical. Memory mapping can present file data through an address range, but it still requires lifetime, bounds and error handling. It does not guarantee zero I/O latency. Avoid assuming that freeing an object immediately reduces the process's visible memory usage: an allocator may retain storage for reuse.

Expanded study

Track four different quantities

An object count, allocator reservation, virtual-address range and resident physical storage are different measurements. Destroying an object does not necessarily release its containing allocation, and releasing an allocation does not necessarily cause the runtime to return pages immediately. A memory-mapped file gives an addressable view of file bytes; it does not create a collection of C++ objects merely by assigning a pointer type. [R16, P07]

Read-only mapping has a lifetime

The reference opens a small regular file, maps it read-only, computes a digest while the mapping is live, then closes the mapping before leaving the file context. No view escapes to a later task. An empty file follows a special path because mapping zero bytes is not a portable way to handle emptiness. The file must remain unchanged during the comparison; a concurrent truncation would violate the lab's environmental assumption. [P07]

Make comparisons about identical content

The streaming path hashes fixed-size chunks. The mapping path passes the mapping to the same digest operation without converting the entire file to a new bytes object. Equal digests are a practical content check for this experiment, and direct small-fixture comparisons can supplement them. This does not establish that either access path performs fewer storage operations: cache warmth, page faults and buffering affect both.

Design a trustworthy timing extension

Create the input outside the timed region and state whether opening, mapping, hashing and closing are included. Change test order across trials. Do not call a second run a cold-cache measurement merely because it uses a different API. This course does not ask the learner to flush system caches, change kernel settings or allocate enough memory to destabilize the workstation. Keep the baseline small and report what was actually controlled.

Worked reasoning

A page-aligned 4100-byte region spans two chosen 4096-byte pages, although only four bytes lie in the second page. That is page-coverage arithmetic, not a resident-set measurement. The demo writes 4096 known bytes and obtains identical streamed/mapped digests. Its useful conclusion is content agreement under the no-concurrent-writer assumption, not a claim that mapping is universally faster.

Runnable example

File: `python/examples/lesson_17_2.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.2 --build-dir build
```

```
import tempfile
from python.reference.lab_17_2 import file_digests
with tempfile.TemporaryDirectory() as d:
    path=Path(d)/"asset.bin"
    path.write_bytes(bytes(range(256))*16)
    a,b=file_digests(path)
    print("equal=",a==b," bytes=",path.stat().st_size,sep="")
```

Expected output from the supplied reference in the validated environment:

```
equal=True bytes=4096
```

Practical checkpoint

Compare sequential file reads with a read-only mapped-file approach on a small disposable file. Pass: verify identical content and report virtual/resident concepts separately without promising one method is faster.

Expanded contract: Python file_digests(path) returns two SHA-256 hex digests for a regular file no larger than 1 MiB. The file resides in a trusted scratch directory and is not modified concurrently. Empty files are supported; no view escapes the mapping context.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_2.py`. Use `python3 check_lab.py 17.2 --student` after implementation.

Independent transfer

An allocator frees 1000 game entities but the process memory graph stays flat. Is that evidence of a leak by itself?

Quiz — answer before opening the key

1. **Does reserving virtual address space imply all pages are resident?**
2. **Why may process memory usage stay high after freeing objects?**
3. **Does memory mapping eliminate storage latency?**
4. **Why does the empty-file case need an explicit branch?**
5. **Why should the mapping be closed before its resources are discarded?**

Assigned study

R16: Address Spaces, Address Translation, Paging and Memory API.

Expansion source map: R16, P07. See the source directory for the exact documents and version notes.

17.3 — Scheduling, responsiveness and real-time constraints

Learning objective: Distinguish average completion from meeting a deadline.

Original foundation

A scheduler chooses runnable work according to its policy. Context switches have costs, and oversubscription can create contention rather than useful parallelism. Throughput-oriented scheduling is not necessarily suitable for a time-critical audio or input path. Real-time reasoning concerns deadlines and worst-case behavior under stated assumptions. General desktop operating systems do not make arbitrary user programs hard-real-time merely because a thread has high priority. Priority inversion occurs when high-priority work waits indirectly on lower-priority work holding a needed resource. Design short critical sections and bounded work before experimenting with priorities.

Expanded study

Define the deadline in the same clock domain

A deadline is a time by which a required result must be ready, not just a hoped-for average duration. The scheduler model uses release time, execution duration and an absolute deadline, all in the same invented time units. Start time cannot precede release. Finish equals start plus duration. Positive lateness is max(0, finish-deadline). This isolates scheduling arithmetic before studying noisy real measurements.

Blocking can dominate a lightly loaded machine

The model is nonpreemptive first-come, first-served. Once a long background job starts, a newly released audio-like job must wait. Average utilization over a long interval might be low while that one wait misses a short deadline. The remedy may involve shorter work units, an appropriate priority policy or eliminating shared blocking resources. Adding workers is not a guaranteed remedy when the bottleneck is one serialized resource. [R16]

Distinguish inversion from ordinary contention

Priority inversion specifically concerns a higher-priority task waiting on a resource held by lower-priority work, potentially while medium-priority work delays the holder. The FCFS example demonstrates blocking latency, not a complete priority-inversion implementation. Explain the three-task resource timeline on paper rather than assigning privileged thread priorities on a primary machine. A standard desktop process is not made hard-real-time by a thread label. [R16]

Choose a structural repair

An audio callback should not perform file loading under a mutex needed by a background importer. One repair is to prepare immutable data elsewhere and publish a ready handle through a bounded handoff. The callback then performs a small predictable operation or uses a fallback if data is unavailable. Such a design still requires a correct ownership protocol; removing a mutex without replacing its lifetime guarantee creates a different failure.

Worked reasoning

Background job B is released at 0, lasts 12 units and has deadline 100. Audio-like job A is released at 1, lasts 1 and has deadline 5. B runs 0..12; A runs 12..13 and is eight units late. A revised admission plan may release a two-unit preparation job at 0, A at 1, and the remaining background work at 3. A then completes at 3. This is a changed model and workload admission policy, not proof that any OS will reproduce the schedule.

Runnable example

File: `python/examples/lesson_17_3.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.3 --build-dir build
```

```
from python.reference.lab_17_3 import schedule_fcfs
for row in schedule_fcfs([(0,12,100),(1,1,5)]):
    print(row)
print("Times above are model units, not measured OS latency.")
```

Expected output from the supplied reference in the validated environment:

```
{'job': 0, 'start': 0, 'finish': 12, 'lateness': 0}
{'job': 1, 'start': 12, 'finish': 13, 'lateness': 8}
Times above are model units, not measured OS latency.
```

Practical checkpoint

Simulate a scheduler or instrument a bounded producer/consumer workload without changing privileged priorities. Pass: show one contention-induced latency spike and propose a structural fix rather than only raising priority.

Expanded contract: Python schedule_fcfs accepts at most 128 jobs (release,duration,absolute_deadline), with bounded nonnegative integer times, positive duration and deadline >= release. It is nonpreemptive, ties use input order, and outputs include start, finish and nonnegative lateness.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_3.py`. Use `python3 check_lab.py 17.3 --student` after implementation.

Independent transfer

A performance report says average frame cost is 4 ms, so every 8 ms deadline must be met. Find the missing evidence.

Quiz — answer before opening the key

1. **What is priority inversion?**
2. **Does low average CPU use guarantee deadlines are met?**
3. **Why avoid blocking I/O in an audio callback?**
4. **A job released at 4 starts at 9, runs for 2 and has absolute deadline 10. What are waiting time and lateness?**
5. **Why is splitting a background task insufficient if all chunks still monopolize a shared lock?**

Assigned study

R16: Scheduling, Multi-Level Feedback Queue and Concurrency Bugs.

Expansion source map: R16. See the source directory for the exact documents and version notes.

17.4 — Filesystems, persistence and asynchronous I/O

Learning objective: Design data operations that survive partial failure.

Original foundation

A filesystem organizes persistent data and metadata. A successful write call does not always mean data has reached durable storage. Buffering, ordering and crash consistency require explicit treatment when durability matters. A robust save process avoids overwriting the only valid copy with partially generated data.

Asynchronous I/O lets work proceed without blocking the caller, but completion can occur later and fail. Buffers and request objects must remain valid until completion according to the API. Cancellation is a request with a contract, not necessarily immediate erasure of all work. Separate correctness, durability and performance goals.

Expanded study

Separate visibility from durability

A save operation has several milestones: valid data in memory, bytes written to a temporary file, buffers flushed, replacement of the destination name, and whatever additional persistence operations the platform requires. An operation returning from write is not the same event as survival of a power loss. The reference performs file flush/fsync and replacement but does not promise parent-directory synchronization or a complete storage-device failure guarantee. [R16, P06]

Preserve the last usable state before commitment

Validate the complete small save schema before touching the destination. Create a unique temporary file in the same trusted directory, write the candidate there, close it and replace the destination. A failure before replacement leaves the previous destination available. Clean up abandoned temporary files owned by this operation. The directory is assumed private and there is one writer; this is not a defense against hostile concurrent path replacement.

Treat post-commit errors differently

An exception injected after replacement occurs after the new file has become visible. Reporting failure does not make the old file reappear. Recovery must inspect the actual committed state rather than blindly retrying a non-idempotent effect. For this save format, replacing with the same validated snapshot is naturally repeatable. The learner must still explain the distinction between a rejected candidate and an uncertain response after commit.

An asynchronous request must own its inputs

The added write_save_async helper takes a private validated snapshot before placing blocking file work on an executor. The caller can change its original dictionary afterward without altering the worker's snapshot. Keep only one outstanding write per destination in this design. A future records completion or failure; it does not eliminate disk latency. This adapter moves blocking work to a thread and is not a kernel-native asynchronous I/O implementation. [P14]

Worked reasoning

Begin with score 7. Inject a failure after half the temporary payload is written: the destination still loads as score 7 and the incomplete temporary file is removed. Now write score 99 and inject after replacement: readback returns 99 even though the caller received an exception. Test both outcomes. A report that calls every exception a rollback would be contradicted by the second experiment.

Runnable example

File: `python/examples/lesson_17_4.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 17.4 --build-dir build
```

```
import tempfile
from python.reference.lab_17_4 import write_save,read_save,InjectedFailure
with tempfile.TemporaryDirectory() as d:
    path=Path(d)/"save.json"
    write_save(path,{"version":1,"score":7})
    try:
        write_save(path,{"version":1,"score":99},"partial_write")
    except InjectedFailure:
        print("injected failure; old score=",read_save(path)["score"],sep="")
    write_save(path,{"version":1,"score":99})
    print("new score=",read_save(path)["score"],sep="")
```

Expected output from the supplied reference in the validated environment:

```
injected failure; old score=7
new score=99
```

Practical checkpoint

Build a save pipeline with injected failures before, during and after serialization. Pass: recover a valid prior or new save under your documented model and never reuse an asynchronous buffer before completion.

Expanded contract: Python write_save/read_save use exactly version=1 and integer score in 0..1000000. write_save supports four named injection points. Readback is capped at 1024 bytes. Guarantees cover ordinary process exceptions on a trusted single-writer scratch path, not power-loss or adversarial-filesystem safety.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_17_4.py`. Use `python3 check_lab.py 17.4 --student` after implementation.

Independent transfer

The caller times out waiting for an asynchronous save future. May it immediately reuse a mutable buffer that the worker might still read?

Quiz — answer before opening the key

1. **Does every successful write imply durable storage?**
2. **Why retain an asynchronous I/O buffer until completion?**
3. **What should a save-recovery test inject?**
4. **What state is expected after the reference injects after_replace?**
5. **Why create the temporary file in the destination directory?**

Assigned study

R16: Files and Directories, Crash Consistency and Journaling.

Expansion source map: R16, P06, P14. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Create a systems lab report covering processes, memory, scheduling and file I/O. Use unprivileged programs and disposable data; do not alter kernel or device settings on a primary machine.

Use only ordinary user privileges and disposable files. Compare execution and storage mechanisms while keeping the contents and useful work constant.

1. A process/thread ownership table, a child-process run and a reported worker error.
2. Identical stream/mapping digests, the empty-file case, bounds and map lifetime notes.
3. A scheduler trace with an explicit missed deadline; label all synthetic times as model units.
4. Before/during/after replacement save-failure tests, including observed old/new destination state.
5. An asynchronous offload lifetime diagram and completion/error check; no power-loss or hard-real-time claim.

Live review: An asynchronous save returns an exception after replacement. Explain why retry/recovery must inspect state rather than assuming that no write occurred.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.

Module 18: Concurrency and parallel programming

Original prerequisites: Modules 12 and 16-17.

Integration goal: Bounded native job system. Integrate the native queue and finite task graph. Use independent output slots and explicit dependencies rather than one giant lock around useful computation.

18.1 — Races, mutexes and happens-before

Learning objective: Protect shared invariants using a clear synchronization model.

Original foundation

Concurrent execution interleaves operations. A C++ data race occurs when conflicting accesses are not properly ordered and at least one is a write, with the relevant accesses not atomic; this leads to undefined behavior. A race condition is a broader logical timing defect and can exist even when individual accesses are atomic. A mutex protects a shared invariant only when all relevant participants follow the same locking protocol. The critical section must cover the complete operation that needs to be atomic at the application level. Ownership and message passing can reduce shared mutable state. Document the happens-before relationships rather than relying on "it usually runs first."

Expanded study

Separate language races from logical races

A C++ data race concerns conflicting memory accesses without the synchronization required by the language. A logical race can remain even when each individual field access is atomic. For example, reading a sufficient balance and later debiting it are separate operations unless one protocol protects the whole decision. Testing repeatedly on one machine cannot replace the language-level ordering argument. [R11]

Protect an invariant rather than a variable name

The Inventory example has two fields: gold and stock. A successful purchase must reduce both consistently, while a rejected purchase changes neither. All reads participating in that state, including a diagnostic snapshot, follow the same mutex protocol. Returning gold and stock through two unrelated unlocked getters would permit a report that combines values from different moments. The mutex belongs with the state so callers cannot easily forget the relationship.

Locate the linearization point

Each buy call checks affordability and availability, applies the updates and releases the mutex. Relative to other callers following the protocol, the operation behaves as one indivisible purchase. The actual lock interval is small and contains no file or network I/O. A snapshot also acquires the mutex, so it observes a coherent pair. The synchronization guarantee comes from the mutex operations, not from an expectation that one thread is scheduled first. [R11, R16]

Use tests to challenge the implementation

Stress tests check counts and conservation after joining all threads. Tests should include exact funds, no stock, insufficient funds and invalid prices. A race detector can reveal certain erroneous executions, but a clean report does not establish all paths, compiler targets or progress properties. Do not run intentionally undefined non-atomic C++ races as the oracle for a correct answer; illustrate a lost update with a deterministic interleaving table instead. [P05]

Worked reasoning

Start with gold=100 and stock=20. Four threads each attempt twenty purchases at price 10. At most ten purchases can succeed; the final coherent state must be gold=0, stock=10. Which thread succeeds is unspecified and irrelevant to this invariant. By contrast, a naive check-then-subtract interleaving could let two threads both approve spending the same last ten gold. The correct test asserts the invariant, not one scheduling order.

Runnable example

File: `native/examples/lesson_18_1.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.1 --build-dir build
```

```
#include "lab_18_1.hpp"
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>
int main() {
    course::Inventory inventory(100,20);
    std::atomic<int> sold{0};
    std::vector<std::jthread> workers;
    for (int i=0;i<4;++i) {
        workers.emplace_back([&]{
            for(int j=0;j<20;++j) {
                if(inventory.buy(10)) sold.fetch_add(1);
            }
        });
    }
    for(auto& thread:workers) thread.join();
    const auto [gold,stock]=inventory.snapshot();
    std::cout<<"sold="<<sold.load()<<" gold="<<gold<<" stock="<<stock<<'\n';
}
```

Expected output from the supplied reference in the validated environment:

```
sold=10 gold=0 stock=10
```

Practical checkpoint

Implement a shared counter and a compound inventory transaction with proper synchronization. Pass: stress test, inspect with an available race detector and explain why detector silence alone is not proof.

Expanded contract: Native Inventory(gold,stock) rejects negative initial values. buy(price) requires price>0; it changes both fields atomically under its mutex or changes neither. snapshot() returns a coherent pair. Destruction occurs only after users have joined.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_1.hpp`. Use `python3 check_lab.py 18.1 --student` after implementation.

Independent transfer

Both gold and stock are atomic integers. Is that enough to make a purchase across both fields transactional?

Quiz — answer before opening the key

1. **Can individually atomic reads/writes still form a logical race?**
2. **What must a mutex protocol protect?**
3. **Why is "this thread usually runs first" insufficient?**
4. **Why does snapshot() also lock?**
5. **Does the order of successful buyer threads need to match between test runs?**

Assigned study

R11: Concurrency and Parallelism; R16: Locks and Concurrency Bugs.

Expansion source map: R11, R16, P05. See the source directory for the exact documents and version notes.

18.2 — Condition variables, deadlocks and cancellation

Learning objective: Make waiting and shutdown correct under adverse timing.

Original foundation

A condition variable allows a thread to wait for a predicate to become true while coordinating with a mutex. Wait in a loop because wakeups do not by themselves prove the predicate. Notifications and state changes must follow a consistent protocol to avoid missed progress. Deadlock can arise from circular resource dependencies. A lock-order rule or reduced lock scope can prevent common cases. Shutdown deserves the same rigor as steady-state execution: wake waiters, stop accepting work, define what happens to queued jobs and join threads safely. Cancellation must specify whether work is abandoned, completed or rolled back.

Expanded study

Write the queue state machine

The bounded queue owns fixed storage, head, tail, a size count and a closed flag. Its invariant is 0 <= size <= capacity. The producer may write only when space exists and the queue is not closed. The consumer may read only when data exists. Size disambiguates empty and full when head and tail happen to be equal after wrapping. Keeping storage fixed after construction makes the queue's own capacity easy to state.

Wait on state, not on a notification

The producer waits for closed OR size<capacity. The consumer waits for closed OR size>0. The predicates are checked while holding the same mutex that protects updates. A condition-variable wait releases the mutex while sleeping and reacquires it before checking again. A notification is a hint to recheck, not ownership of an item or a guarantee that another consumer has not already taken it. [R11, R16]

Make closure a supported transition

Closing with drain=true rejects new pushes while allowing already accepted values to be popped. Once closed and empty, pop returns no value. Closing with drain=false discards values still queued; a value already popped belongs to its consumer and cannot be revoked by clearing the queue. Close notifies both producers waiting for space and consumers waiting for data. Leaving either group asleep can make shutdown hang.

Join only after releasing the resources workers need

The owner stops producers, closes the queue according to policy, lets consumers observe the transition and joins their threads. It must not hold the queue mutex while joining a consumer that needs that mutex to finish. Destruction of a queue with live waiting threads is not made safe by having a destructor. The reference deliberately requires the owner to arrange closure and joining; this responsibility belongs in the ownership map. [R11]

Worked reasoning

Capacity is two. Push 7 and 8, then close in drain mode. The next pops return 7, 8 and no value; a later push returns false. Under discard mode, closing before either pop causes the next pop to return no value immediately. In both cases, accepted-but-unprocessed work must be accounted for by the application. A queue closing successfully does not imply every accepted task completed successfully.

Runnable example

File: `native/examples/lesson_18_2.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.2 --build-dir build
```

```
#include "lab_18_2.hpp"
#include <iostream>
#include <thread>
#include <vector>
int main() {
    course::BoundedQueue queue(2);
    std::vector<int> seen;
    std::jthread consumer([&]{
        while(auto value=queue.pop()) seen.push_back(*value);
    });
    for(int value=1;value<=5;++value) queue.push(value);
    queue.close();
    consumer.join();
    for(auto value:seen) std::cout<<value<<' ';
    std::cout<<'\n';
}
```

Expected output from the supplied reference in the validated environment:

```
1 2 3 4 5
```

Practical checkpoint

Create a producer/consumer queue with graceful shutdown and failure propagation. Pass: test stopping with an empty queue, a full queue and active consumers; no thread remains blocked indefinitely.

Expanded contract: Native BoundedQueue(capacity) accepts 1..65536. push waits for space or closure and returns false after closure. pop waits for data or closure and returns optional<int>. close(true) drains; close(false) discards pending entries. Already popped work is not cancelled, and the owner must join users before destruction.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_2.hpp`. Use `python3 check_lab.py 18.2 --student` after implementation.

Independent transfer

A stop flag is set but an empty-queue consumer never wakes. What must change?

Quiz — answer before opening the key

1. **Why test a condition-variable predicate in a loop?**
2. **What does a consistent lock order prevent?**
3. **Why must shutdown notify blocked workers?**
4. **Why must closing a full queue notify producers as well as consumers?**
5. **What happens to work already removed by a consumer when close(false) runs?**

Assigned study

R16: Condition Variables, Semaphores and Concurrency Bugs; R11: locking guidance.

Expansion source map: R11, R16. See the source directory for the exact documents and version notes.

18.3 — Atomics, memory ordering and false sharing

Learning objective: Use low-level synchronization only with a complete correctness argument.

Original foundation

Atomic operations provide indivisible access according to their type and memory-order rules. Acquire/release synchronization can publish earlier writes when the specified read observes the release sequence appropriately. Relaxed ordering does not provide the same cross-object publication guarantees. Volatile is not a substitute for C++ thread synchronization. False sharing occurs when independent frequently written data occupy a shared cache line, causing coherence traffic. Padding may help a measured case but increases footprint. Lock-free algorithms also face reclamation and progress issues; an atomic pointer alone does not make referenced storage safe to free. Prefer a proven library design unless low-level implementation is the explicit learning goal.

Expanded study

Draw the complete publication chain

The one-shot example first writes a non-atomic payload, then stores true to ready with release ordering. The consumer loads ready with acquire ordering until it observes that publication, and only then reads payload. This connects the producer's earlier write to the later read through a specified synchronization relationship. Reading the initial false value does not establish that publication. [R11]

Restrict the protocol to what was proved

There is one producer, one consumer and one publication. The producer does not modify payload after setting ready. The flag is not reset and the storage remains alive until both participants finish. Reusing the same flag for many messages adds ownership and generation problems that this proof does not solve. A consumer reading an old true flag while a new payload is being written would not be following the one-shot contract.

Choose weaker ordering only for a reason

A relaxed atomic counter can count independent events when it is not being used to publish other data. That does not make a relaxed ready flag sufficient for a non-atomic payload. Similarly, volatile does not supply C++ inter-thread synchronization. Joining a worker also establishes completion relationships, but the publication example deliberately reads before the explicit join to teach the acquire/release chain. [R11]

Separate false sharing from a data race

Two independent atomic counters may be logically safe but contend when their storage shares a coherence block. The benchmark compares compact counters with a type aligned to a chosen 64 bytes. That number is an experimental layout choice, not measured cache-line information; a target with different granularity can behave differently. Thread startup and scheduling are included in the measured interval. Padding changes both sharing and memory footprint, so report the tradeoff instead of calling padding universally faster. [R15]

Worked reasoning

The required ordering is payload write -> release store -> acquire load observing true -> payload read. For input 42, the consumer returns 42. The reference creates fresh storage on every call; running it one hundred times means one hundred independent one-shot experiments, not a reusable-channel proof. The separate false-sharing experiment uses relaxed atomics for independent increments and reads final values only after joining both writers.

Runnable example

File: `native/examples/lesson_18_3.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.3 --build-dir build
```

```
#include "lab_18_3.hpp"
#include <iostream>
int main(){std::cout<<"published="<<course::publish_once(42)<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
published=42
```

Practical checkpoint

Write a publication litmus test with a documented happens-before diagram. Measure a padded versus unpadded counter layout. Pass: do not claim stress tests prove memory-model correctness or assume a fixed cache-line size without labeling it.

Expanded contract: Native publish_once(uint64_t) creates one publication, returns the published value and joins the producer before local storage dies. The exercise requires a correct acquire/release or stronger protocol; stress-test success alone is insufficient.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_3.hpp`. Use `python3 check_lab.py 18.3 --student` after implementation.

Independent transfer

A developer clears ready and writes another payload immediately after publication. Does the original proof still apply?

Quiz — answer before opening the key

1. **Is volatile a C++ inter-thread synchronization mechanism?**
2. **What is false sharing?**
3. **Why is freeing a node difficult in lock-free structures?**
4. **Why is a relaxed counter acceptable in the padding benchmark but not a drop-in ready flag replacement?**

5. **Does alignas(64) prove two counters cannot false-share on every target?**

Assigned study

R11: atomics and memory model; R16: concurrency; R37: later GPU memory-model comparison.

Expansion source map: R11, R15, R16, P05. See the source directory for the exact documents and version notes.

18.4 — Task graphs, work stealing and scaling limits

Learning objective: Parallelize independent work and measure end-to-end benefit.

Original foundation

A task graph expresses jobs and dependencies. Parallel execution is possible among ready tasks that do not conflict. Work stealing distributes queued tasks dynamically, but scheduling overhead can exceed the benefit of tiny jobs. Batch work at a granularity that matches the workload and hardware. Amdahl's law models the limit from a serial fraction: speedup = 1/(s+(1-s)/p) under its simplifying assumptions. Communication, contention and load imbalance can reduce real speedup further. A job system must handle exceptions, ownership, completion and shutdown, not only start threads. Benchmark the actual simulation rather than an empty task loop.

Expanded study

A dependency is a data-availability contract

In the graph, deps[i] lists prerequisites of task i. A task is ready only after those prerequisites have reached a terminal state; it may run only if they succeeded. The graph must be validated before starting workers: unknown indices, self-dependencies, duplicate edges and cycles are rejected. Otherwise a malformed graph can make every worker wait for a prerequisite that never completes.

Calculate work and span separately

Work is the total useful task cost in a chosen model. Span is the longest dependency path. Even with unlimited ideal workers, a computation cannot finish faster than its span. With p ideal workers it also cannot finish faster than work/p. These lower bounds exclude scheduling and communication overhead. The critical_path exercise computes the longest path using topological processing and rejects duration addition that would overflow its uint64_t model.

Read the scheduler actually supplied

The reference executor keeps a finite graph and a shared-state mutex. Workers claim ready tasks, release the mutex before calling task code, then publish done or failed status under the mutex. Dependents of a failed task become blocked rather than running with invalid inputs. Captured result storage must still have disjoint writes or a documented dependency/locking relationship. The reference is a global ready-scan scheduler, not a work-stealing implementation.

Understand the next architecture before adopting it

Work stealing gives workers local queues and permits idle workers to take suitable work elsewhere. It can reduce some contention and balance irregular jobs, but introduces queue ownership, stealing races and more difficult reclamation/progress questions. Learn the simpler executor's correctness first. Cancellation in this package is demonstrated by the bounded queue and Gate D service; the finite graph reference completes healthy work and blocks failed descendants, and does not provide arbitrary interruption of task bodies. [R11, R16]

Worked reasoning

Two independent tasks have durations 2 and 5; a one-unit aggregation depends on both. Work is 8, span is 6, and the ideal two-worker lower bound is max(8/2,6)=6. A serial chain would take 8. The executable example writes 6 and 9 into disjoint slots, then its dependent aggregation produces 15. Worker count may change the execution order of independent tasks but must not change the required aggregate.

Runnable example

File: `native/examples/lesson_18_4.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.4 --build-dir build
```

```
#include "task_graph.hpp"
#include <array>
#include <iostream>
int main(){std::array<int,3> output{0,0,0};
std::vector<course::Task> graph{{{},[&]{output[0]=6;}},{{},[&]{output[1]=9;}},{{0,1},[&]
{output[2]=output[0]+output[1];}}};
auto report=course::execute_graph(graph,2);
for(auto s:report.state)if(s!=course::TaskState::done)return 1;
std::cout<<"result="<<output[2]<<" modeled_span="<<course::critical_path({2,5,1},{{},{},
{0,1}})<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
result=15 modeled_span=6
```

Practical checkpoint

Implement a small task graph for independent entity updates followed by aggregation. Pass: compare 1, 2 and available worker counts, verify results and explain a case where more workers hurt.

Expanded contract: Native critical_path(durations,deps) accepts at most 1024 nodes, rejects inconsistent dimensions, invalid/duplicate edges and cycles, and returns the longest modeled path with checked uint64 arithmetic. execute_graph is a separate worked integration reference, not the function supplied by the student exercise.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_4.hpp`. Use `python3 check_lab.py 18.4 --student` after implementation.

Independent transfer

All workers execute parent jobs that synchronously wait for their own child jobs in the same bounded pool. What failure is possible?

Quiz — answer before opening the key

1. **What is the ideal maximum speedup with a 25% serial fraction?**
2. **Why can very small tasks reduce performance?**
3. **What does a task dependency express?**
4. **For a 20% serial fraction and four ideal workers, what is Amdahl speedup?**
5. **Does the supplied executor implement work stealing?**

Assigned study

R15: Parallel Processing; R16: concurrency; R11: task-oriented concurrency guidance.

Expansion source map: R11, R15, R16. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Implement a bounded job system with dependency handling, shutdown, failure propagation and stress tests. Compare one-thread and multi-thread execution on a real workload.

Integrate the native queue and finite task graph. Use independent output slots and explicit dependencies rather than one giant lock around useful computation.

1. Queue contracts for capacity, rejection, drain, discard and destruction after joined users.
2. A happens-before explanation for the compound inventory operation and one-shot publication.
3. A graph with independent jobs, an aggregation dependency and a deliberately failed predecessor.
4. Stress and shutdown checks with external timeouts; memory/concurrency diagnostics where supported.
5. An equal-work comparison using 1, 2 and a declared available worker count; keep startup/scheduling costs explicit.

Live review: A prerequisite fails while independent work remains ready. Demonstrate that dependent work is blocked, independent work finishes, and shutdown does not wait forever.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.

Module 19: Networks, storage and data services

Original prerequisites: Modules 15 and 17-18.

Integration goal: Local message and data service. Build a loopback or in-process protocol harness and a relational inventory operation. State exactly which behavior belongs to transport, parsing and persistence.

19.1 — Network layers, latency and transport semantics

Learning objective: Choose transport behavior from application requirements.

Original foundation

Networks exchange data through layered protocols. IP provides packet delivery without guaranteeing arrival or order. TCP presents a reliable ordered byte stream, not a stream of application messages. UDP preserves datagram boundaries but does not itself supply delivery, ordering or congestion-control behavior. Applications still need explicit protocols. Latency, jitter, loss and bandwidth describe different constraints. A high-bandwidth link can still have long round-trip delay. Reliable delivery can involve waiting for missing information, which affects real-time design. Use localhost or an authorized lab network for exercises and keep authentication and encryption decisions separate from transport assumptions.

Expanded study

Choose a transport property, not a slogan

TCP gives the application an ordered byte stream, not a matching receive for each send call. UDP datagrams retain boundaries but do not by themselves supply the application's reliability, ordering, congestion response or security policies. A high-capacity link can still have high delay. Keep bandwidth, latency, jitter and loss separate when reasoning about game messages. [R17]

Make a message boundary explicit

The lab uses a four-byte unsigned big-endian payload length followed by that many bytes. Zero and lengths above 4096 are rejected. A receiver first collects the header, validates it, then collects the bounded body. An advertised length of 0xffffffff is an error, not an invitation to allocate almost four gigabytes. The incremental decoder can emit several messages from one chunk and retain an incomplete tail for the next chunk.

Test segmentation without relying on the operating system

The demo passes deliberately split byte chunks directly to FrameDecoder. Its tests try every split point of two concatenated frames. A localhost example also makes several send calls, but the OS is free to coalesce or split those calls differently; that experiment alone does not force a particular receive pattern. Both the parser-level test and the socket integration test are needed for their different evidence. [P09]

State the input and buffering budgets

One incremental feed accepts at most 8192 bytes, and retained incomplete frame data is bounded by the 4096-byte payload contract plus header. The number of returned message objects is also bounded by that accepted chunk, but Python object overhead remains separate from wire-byte counts. A clean end-of-stream is valid only when no partial frame remains. Once a framing error occurs, the reference decoder is unusable; it does not guess where a new header might start.

Worked reasoning

The frame for the ASCII bytes abc is 00 00 00 03 61 62 63 in hexadecimal. Two frames can be concatenated without a separator because each prefix gives the next body length. Receiving only 00 00 and then the rest is valid fragmentation; reaching EOF after a complete header and only one of three payload bytes is truncation. The socket reader distinguishes EOF from deadline expiration and invalid advertised length.

Runnable example

File: `python/examples/lesson_19_1.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.1 --build-dir build
```

```
from python.reference.lab_19_1 import FrameDecoder,encode_frame
wire=encode_frame(b"one")+encode_frame(b"two")
decoder=FrameDecoder();result=[]
for chunk in [wire[:2],wire[2:6],wire[6:]]:
    result.extend(decoder.feed(chunk))
decoder.finish()
print(result)
```

Expected output from the supplied reference in the validated environment:

```
[b'one', b'two']
```

Practical checkpoint

Create a localhost client/server exchange with a length-prefixed message. Pass: split one message across several writes and combine several messages in one write; the receiver reconstructs them correctly.

Expanded contract: Frames contain a 4-byte network-order length and 1..4096 payload bytes. FrameDecoder.feed accepts bytes chunks <=8192 and emits complete bodies; finish rejects a partial tail. recv_frame reads one frame by an absolute monotonic deadline. Its socket timeout is adjusted as part of the operation.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_1.py`. Use `python3 check_lab.py 19.1 --student` after implementation.

Independent transfer

Your server receives two commands in one recv call. Is TCP broken?

Quiz — answer before opening the key

1. **Does TCP preserve application message boundaries?**
2. **Does high bandwidth imply low latency?**
3. **What behavior must an application add when using UDP?**
4. **What is the seven-byte wire frame for abc?**
5. **Why close a decoder after an invalid length rather than scanning for another apparent prefix?**

Assigned study

R17: TCP service and interface semantics; R16: networking context.

Expansion source map: R17, P09. See the source directory for the exact documents and version notes.

19.2 — Protocol design, serialization and timeouts

Learning objective: Treat every received byte as untrusted structured input.

Original foundation

A protocol defines message types, fields, ranges, versioning and permitted sequences. Serialization must specify byte order and numeric representation. Validate lengths before allocation and cap total resource use. A checksum can detect some corruption but is not equivalent to cryptographic authentication. Timeouts handle missing progress, not certainty about remote state. A timed-out operation may have executed remotely even if its response was lost. Request IDs and idempotent operations help manage retries. Compatibility is a design decision: unknown fields and versions should be accepted, ignored or rejected according to a documented rule.

Expanded study

Validate a complete request contract

Version 1 has exactly four fields: version, id, operation and values. The operation is sum_squares; values is a list of 1..256 integers in 0..1000; IDs contain 1..32 ASCII letters, digits, underscore or hyphen. Booleans are deliberately rejected as numeric values even though Python bool is related to int. Unknown fields and versions are errors. These are chosen protocol rules, not claims that every game should use this schema.

Make parsing ambiguity visible

The parser rejects invalid UTF-8, duplicate JSON field names, non-finite constants and inputs outside the byte limit. It catches excessive nesting/parser recursion as an invalid request. A syntactically valid JSON object is not automatically a valid job. In particular, a client cannot choose an executable, path, arbitrary expression or number of worker threads. Those capabilities are absent from the protocol rather than filtered through a shell later.

Use one absolute deadline

Setting a fresh full timeout for every fragment can let a peer extend total wait time by trickling bytes. The reference calculates remaining=deadline-monotonic_now before each read, so the entire frame shares one budget. A timeout after submission still does not establish whether work committed. The gateway reports UNKNOWN_OUTCOME and tells the caller to retain the same ID and payload when retrying. [P09]

Bind idempotency identity to the work

The store compares a canonical validated request with the stored request for that ID. A repeated ID with identical work may reuse the result; the same ID with different values is rejected. Deduplication has a retention scope: deleting the record can remove protection for a future retry. The reference caps history and rejects new unique IDs when full rather than silently evicting old evidence. This bounds the exercise while keeping its stated replay guarantee simple.

Worked reasoning

Request {version:1,id:job-1,operation:sum_squares,values:[2,3]} represents result 13. Reordering JSON object fields should not change the canonical request. Replacing 2 with true must fail type validation. Retrying job-1 with [2,3] after a lost response should not create another durable record. Retrying job-1 with [2,4] is a conflict, not an update to the original job.

Runnable example

File: `python/examples/lesson_19_2.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.2 --build-dir build
```

```
from python.reference.lab_19_2 import parse_request,canonical_request
raw=b'{"version":1,"id":"job-1","operation":"sum_squares","values":[2,3]}'
print(canonical_request(parse_request(raw)))
try:
    parse_request(raw.replace(b'[2,3]',b'[true,3]'))
except ValueError:
    print("boolean value rejected")
```

Expected output from the supplied reference in the validated environment:

```
{"id":"job-1","operation":"sum_squares","values":[2,3],"version":1}
boolean value rejected
```

Practical checkpoint

Add versioned framing, maximum sizes, timeout handling and request IDs to the lab protocol. Pass: reject oversized and truncated messages before dangerous allocation and test duplicate delivery.

Expanded contract: Python parse_request(bytes) accepts only the exact bounded version-1 schema. canonical_request validates again and emits deterministic sorted compact JSON. IDs do not authorize actions; the local gateway intentionally supplies neither public authentication nor encryption.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_2.py`. Use `python3 check_lab.py 19.2 --student` after implementation.

Independent transfer

The client timed out and invents a new ID for the retry. What guarantee has it lost?

Quiz — answer before opening the key

1. **Does a timeout prove the remote operation never happened?**
2. **Why validate a length before allocating its buffer?**
3. **What does idempotence mean?**
4. **Why reject bool values explicitly in a Python integer schema?**
5. **What does the reference do when result history is full?**

Assigned study

R17: transport failure context; R18: transaction and data-integrity foundations.

Expansion source map: R17, P09, P10, P12. See the source directory for the exact documents and version notes.

19.3 — Relational modeling, SQL and indexes

Learning objective: Represent game data with constraints that survive application mistakes.

Original foundation

A relational schema defines tables, keys and relationships. A primary key identifies a row; a foreign key constrains references between tables. Normalization reduces certain redundancy and update anomalies, while deliberate denormalization trades simpler reads for additional consistency work. SQL queries express desired results, but their execution plans determine physical work. An index can accelerate suitable lookups while consuming storage and adding write cost. Parameterized queries separate data from executable query structure. Do not build SQL by concatenating untrusted text, and do not assume an index helps every query.

Expanded study

Put relationships in the schema

The lab separates players from inventory. A primary key identifies a player. An inventory row references that player and has a composite key (player_id,item_id), preventing two aggregate rows for the same player/item pair. A quantity constraint rejects negative or out-of-contract amounts even if application code makes a mistake. Schema constraints complement validation; neither replaces the other. [R18]

Know which database you are actually running

The executable exercises use SQLite through Python's standard library so no server installation is required. Each connection explicitly enables foreign-key enforcement and uses explicit BEGIN/COMMIT control. PostgreSQL remains the assigned relational reading, and a separate SQL companion supports a PostgreSQL exercise. Do not copy SQLite PRAGMAs, locking behavior or placeholder syntax into a PostgreSQL report and call them identical. [P10, P11, P12]

Separate data from query syntax

All values in the reference are passed as parameters. An unusual player name containing quotes remains one data value; it is not concatenated into a command. Parameterization does not validate gameplay rules, and it does not turn arbitrary user-selected table names into safe values. The schema and supported operations stay under application control. A query should request only the fields needed for its purpose. [R18, P10]

Explain an index using the query it supports

The composite inventory key supports a lookup whose leading field is player_id. A separate item_id index can help item-focused queries, depending on selectivity and the optimizer. Indexes consume storage and maintenance work. Inspect EXPLAIN output and verify the query result before and after adding an index. An optimizer may rationally choose a scan for a tiny table, so seeing no index use is not automatically a database defect. [R18]

Worked reasoning

Seed player A with 20 ore and B with 3 ore. A query ordered by player_id returns [(A,20),(B,3)]. An inventory insert for a nonexistent player fails the foreign-key constraint. A second (A,ore) aggregate row fails uniqueness. Updating quantity to -1 fails the domain constraint. These tests deliberately exercise the database directly rather than relying only on an application helper that may reject the input first.

Runnable example

File: `python/examples/lesson_19_3.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.3 --build-dir build
```

```
from python.reference.lab_19_3 import connect_store,seed_inventory
import sqlite3

db=connect_store()
try:
    seed_inventory(db)
    print(db.execute("SELECT player_id,quantity FROM inventory ORDER BY player_id").fetchall())
    try:
        db.execute("INSERT INTO inventory VALUES (?,?,?)",("missing","ore",1))
    except sqlite3.IntegrityError:
        print("foreign key rejected")
finally:
    db.close()
```

Expected output from the supplied reference in the validated environment:

```
[('A', 20), ('B', 3)]
foreign key rejected
```

Practical checkpoint

Create a small schema with primary, foreign and uniqueness constraints. Use parameterized queries and inspect one indexed query plan. Pass: demonstrate rejected invalid references and compare a query before/after indexing.

Expanded contract: Python connect_store creates players, inventory and operations tables and enables foreign keys. seed_inventory atomically creates A/B and ore quantities 20/3. Explicit SQL controls transactions. A connection belongs to its thread and must be closed by its owner.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_3.py`. Use `python3 check_lab.py 19.3 --student` after implementation.

Independent transfer

A tiny-table query ignores a newly created index. Must the learner force index use to pass?

Quiz — answer before opening the key

1. **What does a foreign key protect?**
2. **Why can indexes slow writes?**
3. **Why use parameterized SQL?**
4. **Why enable SQLite foreign keys on each connection?**
5. **Why use a composite uniqueness constraint on player_id and item_id?**

Assigned study

R18: tables, queries, joins, foreign keys and indexes via the linked manual.

Expansion source map: R18, P10, P11, P12. See the source directory for the exact documents and version notes.

19.4 — Transactions, consistency and distributed failure

Learning objective: Preserve invariants when several operations must succeed together.

Original foundation

A transaction groups related database work under defined atomicity, consistency, isolation and durability guarantees. Isolation levels differ; applications must understand which anomalies remain possible. A transfer should not debit one account without the corresponding credit. Concurrency can require retries even when each statement is individually correct. Across multiple services, a local transaction does not automatically create a global one. Partial failure, duplicate messages and stale reads require explicit coordination. Separate durable authoritative data from caches. A cache miss is recoverable; losing the only authoritative record is a different failure. Test the system's invariants under interruption rather than only normal response codes.

Expanded study

Choose the invariant before writing updates

A transfer of ore between two players must preserve total ore, keep both quantities valid and either apply both sides or neither. A transaction groups the debit, credit and replay record into one commit. A database can enforce the local transaction boundary; it cannot automatically include effects in another service or a client display. Record the authority boundary explicitly. [R18]

Make retries part of the transaction

The operations table stores operation ID, canonical payload and result. The transaction first checks for an existing ID. An identical payload returns the stored response; a conflicting payload is rejected. For new work, it verifies players, available quantity and destination capacity, applies both changes, and inserts the replay record before commit. Storing the replay record after a separate commit would create a window in which a retry could repeat the effect.

Understand the chosen isolation mechanism

The SQLite reference uses BEGIN IMMEDIATE to establish its write transaction before the read/check/update sequence. A competing writer may wait or report a busy condition. This simplifies the single-database exercise but is not a claim about PostgreSQL row-level locking or every isolation level. PostgreSQL study should cover the chosen isolation level and its retryable failures using that server's own documentation. [P12, R18]

Inject on both sides of commitment

The test suite raises after debit, before commit and after commit. The first two paths roll back all transaction changes. The final path simulates losing a response after persistence: it must not attempt to reverse a completed transaction merely because the caller did not receive the reply. A restart using the same durable database can still recognize a repeated operation. Replacing the database with a fresh one loses that history and is outside the replay guarantee.

Worked reasoning

Start at (A=20,B=3). A five-ore transfer produces (15,8), preserving total 23. Failure after the debit must leave (20,3) with no operations row. Failure after commit leaves (15,8) plus one operation record. Retrying the same ID and payload returns the original response and remains at (15,8). Retrying the same ID with quantity 6 is rejected without changing inventory.

Runnable example

File: `python/examples/lesson_19_4.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 19.4 --build-dir build
```

```
from python.reference.lab_19_3 import connect_store,seed_inventory
from python.reference.lab_19_4 import transfer,ResponseLost
db=connect_store()
try:
    seed_inventory(db)
    try:
        transfer(db,"op1","A","B","ore",5,"after_commit")
    except ResponseLost:
        print("reply lost; retry same request")
    print(transfer(db,"op1","A","B","ore",5))
    print("total=",db.execute("SELECT SUM(quantity) FROM inventory").fetchone()[0],sep="")
finally:
    db.close()
```

Expected output from the supplied reference in the validated environment:

```
reply lost; retry same request
{'destination_quantity': 8, 'source_quantity': 15}
total=23
```

Practical checkpoint

Implement an inventory transfer with transactions and forced failure between updates. Pass: verify conservation, rollback and duplicate-request handling; document what is not guaranteed across processes or services.

Expanded contract: Python transfer requires distinct known players, valid identifiers and quantity 1..1000000. It conserves quantity, enforces destination capacity, persists at most 1024 replay records, and rejects conflicting ID reuse. ResponseLost after_commit represents committed work with a lost reply, not rollback.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `python/student/lab_19_4.py`. Use `python3 check_lab.py 19.4 --student` after implementation.

Independent transfer

The debit is committed in database X and the credit is sent to service Y, which times out. Is a local SQL transaction enough?

Quiz — answer before opening the key

1. **What does transaction atomicity mean?**
2. **Does a transaction in one database automatically include another service?**
3. **What invariant should an item-transfer test preserve?**
4. **Why insert the replay record in the same transaction as the effect?**
5. **Does a caught after_commit error imply total quantity changed incorrectly?**

Assigned study

R18: Transactions; R16: persistence and failure reasoning.

Expansion source map: R18, R16, P10, P12. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Build a local-only client/server message exercise and a small relational game-data service. Test malformed framing, transaction rollback and duplicate requests without exposing a public service.

Build a loopback or in-process protocol harness and a relational inventory operation. State exactly which behavior belongs to transport, parsing and persistence.

1. A four-byte big-endian framing specification, fragmented/coalesced parser checks and a truncated-EOF test.
2. An absolute receive deadline, byte/field bounds and strict versioned schema validation.
3. Primary/foreign/unique/check constraints, parameterized queries and a before/after index plan.
4. Atomic transfer with injected failure, conserved quantity and an operation replay record in the same transaction.
5. A lost-response retry and conflicting-ID case; retain history and state the replay window.

Live review: The first reply is lost after commit, then the same ID arrives with a different payload. Show that the old effect remains and the conflicting request is rejected.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.

Module 20: Engineering practice, security and maintainability

Original prerequisites: Volumes 01-04.

Integration goal: Gate D evidence and release. Assemble the earlier investigations into the local workload-processing application. Complete the Gate D specification rather than adding a public deployment.

20.1 — Requirements, architecture and design records

Learning objective: Make tradeoffs explicit and trace implementation back to needs.

Original foundation

A requirement states an observable need or constraint. Functional requirements describe behavior; quality requirements describe properties such as latency, reliability or accessibility. Architecture allocates responsibilities and dependencies to meet those requirements. A diagram without contracts and failure behavior is not enough. An architecture decision record states the problem, constraints, alternatives, chosen approach and consequences. Avoid premature distributed systems or custom frameworks when a smaller design meets the need. Revisit decisions when evidence changes. Traceability links requirements to tests and avoids declaring success merely because all planned code was written.

Expanded study

A requirement needs an observation boundary

"Fast" and "reliable" do not identify what to measure. A useful requirement describes the workload, environment, result and acceptance rule. For ForgeQueue, an example is: reject an advertised body length above 4096 before allocating that body. Another is: with capacity four, no more than four accepted jobs may be queued or running inside the service. State separately whether completed results retained by a caller are inside that budget.

Map responsibilities before distributing them

The reference separates framing, request validation, admission, native computation and persistence. Python handles the local orchestration; a trusted C++ executable performs the bounded arithmetic. The small workload deliberately makes overhead visible. It does not establish that launching a process is a good per-frame engine architecture. The C++ queue and task-graph labs teach in-process concurrency separately. Choosing a boundary means accepting its copying, lifetime and failure costs.

Write a decision that can be revisited

An architecture decision record includes context, constraints, alternatives, the selected option and consequences. For example, a SQLite result store avoids requiring a server for this local learning project, but it narrows the concurrency and deployment assumptions. A no-network file-driven interface may be even simpler; a loopback gateway is useful when the learning objective includes framing and timeouts. A rejected alternative is not universally bad; it was not selected under these constraints.

Trace requirements to evidence

The requirements sheet names the implementation boundary and an acceptance test for each claim. A test name is not a test result, and a green pipeline cannot establish requirements it never exercised. Include reviewer judgment for ownership arguments, threat assumptions and accessibility observations. Keep the source program's Gate D requirements intact while marking the new ForgeQueue design as one worked realization, not the only permissible student project.

Worked reasoning

A complete requirement might read: "REQ-3: On a retry with the same validated ID and identical payload against the same retained database history, return the committed response without inserting a second result row." Acceptance evidence includes a direct repeat, concurrent repeats, a restart and a conflict case. Merely checking that two calls returned HTTP-like success text would not establish the durable-row invariant.

Runnable example

File: `python/examples/lesson_20_1.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.1 --build-dir build
```

```
requirements={"REQ-1":"reject oversized frame","REQ-2":"drain accepted work","REQ-3":"deduplicate
durable result"}
evidence={"REQ-1":["test_oversize"],"REQ-2":[],"REQ-3":["test_replay"]}
missing=[r for r in requirements if not evidence.get(r)]
print("Requirements missing evidence:",missing)
print("Named tests still need to be run and reviewed; names are not proof.")
```

Expected output from the supplied reference in the validated environment:

```
Requirements missing evidence: ['REQ-2']
Named tests still need to be run and reviewed; names are not proof.
```

Practical checkpoint

Write six measurable requirements and two decision records for the simulation. Pass: every requirement has an acceptance test and each decision includes a rejected alternative and a cost.

Expanded contract: Evidence lab: requirements, traceability and two architecture decision records. The demonstration only detects missing evidence entries; it does not certify architecture quality or execute named tests.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

Every planned programming task is complete, but the service accepts unbounded jobs. Is Gate D complete?

Quiz — answer before opening the key

1. **What makes a requirement testable?**
2. **Why record rejected alternatives?**
3. **Is completing planned tasks the same as satisfying requirements?**
4. **Why name the retained database history in a replay requirement?**

5. **What evidence is missing when a requirements matrix contains only implementation filenames?**

Assigned study

R01: Software Engineering; R02: engineering design and professional practice.

Expansion source map: R01, R02, R05. See the source directory for the exact documents and version notes.

20.2 — Defensive programming and threat modeling

Learning objective: Protect assets and trust boundaries before selecting security mechanisms.

Original foundation

A threat model identifies assets, actors, entry points, trust boundaries and plausible failures. Game clients, mods, save files and asset importers are not automatically trusted. Validate input, minimize privileges and isolate risky processing. Memory-safe interfaces, bounds checks, fuzzing and sanitizers help address different defect classes. Authentication establishes an identity or credential; authorization decides permitted actions. Encryption alone does not enforce game rules or protect a compromised endpoint. Use established security libraries rather than inventing cryptography. Perform testing only against your own code or authorized lab targets; the purpose here is defensive engineering.

Expanded study

Identify assets and authority

The reference protects workstation availability, private scratch files and the meaning of stored results. Entry points include request bytes, configuration paths and the configured native executable. A client supplies bounded data, not executable code. The parent directory and native binary are trusted owner configuration. The loopback binding reduces unintended exposure but does not authenticate local users, and the project must not be described as a public-ready server.

Bound more than one resource

A payload length limit does not bound the number of accepted jobs. A worker-count limit does not necessarily bound an executor's waiting queue. ForgeQueue uses a semaphore for queued-plus-running work, fixed payload/value limits, a subprocess wait budget and a durable-history quota. The code assumes the shipped trusted worker emits bounded output; it is not a sandbox for an arbitrary hostile executable that floods output or consumes unlimited memory. Record that residual risk explicitly.

Fuzz within a declared target

The demonstration feeds bounded generated byte strings only to the supplied parser. Random bytes mostly exercise early rejection, so a stronger lab also mutates valid messages: remove fields, duplicate fields, change types, cross length boundaries and preserve enough structure to reach later validation. Keep the seed and the smallest failing input. A fixed input count with no unexpected exception is useful observed evidence, not proof that the parser is secure. [P04, P05]

Give failures useful but restrained messages

An operator needs to distinguish invalid input, overload, worker failure and uncertain outcome. Full request bodies, filesystem paths, credentials and internal stack traces usually do not belong in a client-facing message. Keep detailed diagnostics in a controlled local record when needed. Encryption and authentication would be separate additions for a different deployment; they would not eliminate validation or authorization responsibilities.

Worked reasoning

A four-byte header advertises 4,294,967,295 bytes. The receiver rejects it before requesting a body of that size. A correctly sized request asks for operation "exec"; the parser rejects it because only sum_squares is allowed. A valid request arrives while all four admission tickets are occupied; the service returns BUSY. These failures occur at different boundaries and should not all become one misleading "bad JSON" message.

Runnable example

File: `python/examples/lesson_20_2.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.2 --build-dir build
```

```
import random
from python.reference.lab_19_2 import parse_request
rng=random.Random(2026);rejected=0
for _ in range(500):
    raw=bytes(rng.randrange(256) for _ in range(rng.randrange(129)))
    try:
        parse_request(raw)
    except ValueError:
        rejected+=1
print("bounded random inputs=500 rejected=",rejected,sep="")
print("No observed unexpected exception; not a security proof.")
```

Expected output from the supplied reference in the validated environment:

```
bounded random inputs=500 rejected=500
No observed unexpected exception; not a security proof.
```

Practical checkpoint

Threat-model the save parser and local service. Add bounds checks and a small malformed-input fuzz harness. Pass: cap work/memory, preserve useful errors and record residual risks instead of claiming complete security.

Expanded contract: Evidence lab: threat model plus bounded fuzz/mutation report against this package only. No network scanning, public deployment or untrusted executable execution is required. Claims must distinguish tests, design arguments and untested assumptions.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

The service binds only to 127.0.0.1. Does that authorize every local caller?

Quiz — answer before opening the key

1. **How does authorization differ from authentication?**
2. **Why is a game client a trust boundary?**
3. **Does encryption replace input validation?**
4. **Why does a worker-count limit alone not guarantee bounded backlog?**
5. **What additional fuzz cases reach beyond random-byte rejection?**

Assigned study

R01: Security; R11: bounds/lifetime safety; R02: information-security breadth.

Expansion source map: R01, R02, R11, P04, P05, P09, P14. See the source directory for the exact documents and version notes.

20.3 — Build automation, dependencies and release discipline

Learning objective: Produce a verifiable artifact from a known source revision.

Original foundation

Continuous integration runs repeatable checks when changes are made. A useful pipeline builds, tests and reports failures from a clean environment. Pin dependencies sufficiently to reproduce the result and record toolchain versions. A dependency inventory includes origin, version and license obligations; it is not enough to know that a package downloaded successfully. Separate secrets from source and limit automation permissions. Preserve symbols or diagnostics needed to investigate crashes, while controlling access to sensitive information. A release artifact should be traceable to a commit and build configuration. Automated success is evidence only for the checks actually run.

Expanded study

Define what is reproduced

A clean build should produce a runnable artifact from a known source tree and documented toolchain. That is different from a claim that two builds are bit-for-bit identical. Compiler versions, generated metadata and archive timestamps can affect bytes. The package supplies a source/file manifest and exact verification commands; they identify tested inputs without promising identical binaries across compilers or operating systems. [R05, P13]

Fail before publishing

The local verification script configures the native project, builds it, runs CTest, runs the Python tests and executes the demonstrations. A nonzero result must stop the process. The release helper invokes verification before creating a new archive; its explicit failure-injection option checks that a failed step does not generate a fresh release. Do not suppress errors to make a release button look successful.

Inventory runtime and build dependencies

The code requires Python, its sqlite3 module, a C++20-capable toolchain, CMake and platform thread support. No package download happens during the verification script. Record concrete versions from the actual machine and distinguish them from the document-rendering tools used to author the PDFs. External reading links are not bundled third-party source. A dependency inventory should note provenance and applicable terms rather than inventing legal conclusions.

Treat integrity and authenticity separately

SHA-256 lets a reviewer compare a file with an expected digest. If both the file and its digest are replaced by an untrusted party, that comparison alone establishes no trusted origin. The manifest excludes its own digest to avoid a self-reference loop and states the exclusion. Preserve diagnostic output needed for a failure, but do not accidentally archive secrets, local databases or unrelated home-directory files.

Worked reasoning

Change one byte in native/worker.cpp: its digest changes. That detects a mismatch against the original recorded value. Now intentionally force the verification process to return nonzero. The release helper must exit without creating the requested new ZIP. Finally perform a clean successful run, identify the source tree and toolchain, and extract the release into another directory to rerun the same documented checks.

Runnable example

File: `python/examples/lesson_20_3.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.3 --build-dir build
```

```
import hashlib
source=ROOT/"native"/"worker.cpp"
before=hashlib.sha256(source.read_bytes()).hexdigest()
changed=hashlib.sha256(source.read_bytes()+b"\n").hexdigest()
print("change detected=",before!=changed,sep="")
print("A digest detects byte changes against a trusted value; it is not authentication.")
```

Expected output from the supplied reference in the validated environment:

```
change detected=True
A digest detects byte changes against a trusted value; it is not authentication.
```

Practical checkpoint

Create a local clean-build script that runs tests and produces a versioned archive plus manifest. Pass: run it from a fresh build directory and deliberately break a test to confirm the pipeline stops.

Expanded contract: Evidence lab: clean build/test log, failed-pipeline demonstration, dependency inventory, versioned source release and extraction/rebuild evidence. A hash manifest is an integrity aid, not an authenticity certificate.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

The pipeline printed a failed test but returned exit code zero and created a release. Does it satisfy the acceptance rule?

Quiz — answer before opening the key

1. **What is the purpose of dependency pinning?**
2. **What should identify a release artifact's origin?**
3. **Why test a pipeline with an intentional failure?**
4. **Why does the package manifest exclude its own hash?**
5. **Does the same source hash guarantee identical executable bytes on two compilers?**

Assigned study

R05: workflows and tagging; R12: builds and testing; R01: software engineering.

Expansion source map: R05, R12, P13. See the source directory for the exact documents and version notes.

20.4 — Ethics, accessibility, teamwork and sustainable delivery

Learning objective: Evaluate engineering decisions beyond code correctness.

Original foundation

Engineering work affects players, colleagues and the public. Accessibility, privacy, intellectual property, honest reporting and sustainable maintenance belong in technical decisions. Obtain permission for assets and code and respect the licenses under which they are supplied. Do not assume publicly visible material is free to redistribute. Teams need clear responsibilities, respectful review and realistic scope. Crunch is not a substitute for planning and can hide systemic delivery problems. A postmortem should identify contributing conditions and improvements, not scapegoats. For hardware, safety and environmental constraints become additional design requirements, not optional polish.

Expanded study

Evaluate the operator experience as part of correctness

A service may calculate the right result and still be difficult to use when its status messages are ambiguous or its recovery instructions are missing. The reference uses textual status and stable error codes rather than color alone. A learner should test small-window/terminal use, keyboard-only operation where relevant and whether a second person can recover from an ordinary input mistake. These are review activities, not outcomes that a unit test suite can fully certify. [R29]

Keep privacy and attribution concrete

Use generated or consented data for the course. A bug report usually does not need a player's real name, credentials or an entire private save file when a small synthetic reproduction demonstrates the defect. Record which code is original student work, which is supplied reference material and which external components are used. Public access to a resource is not a reason to omit its provenance or applicable permissions. [R01, R02]

A retrospective explains contributing conditions

An incident review should name the observed failure, its impact, detection, timeline, contributing design or process conditions, and specific improvements. "The developer was careless" does not describe a reproducible mechanism or prevention check. A better explanation might be that cancellation tests never included a queued job and the release process did not exercise shutdown under load. That finding suggests an owner, a new test and an acceptance rule.

Finish at an honest scope

Gate D is a small defensible systems application, not a commercial online backend. Preserve unresolved assumptions and platform limits in the handoff. A smaller working scope with clear evidence is preferable to adding public hosting, arbitrary plugins or extra protocols without corresponding tests and review. Passing the code checks does not replace a live explanation, independent extension or the learner's own failure investigation.

Worked reasoning

Replace "ERROR: invalid" with "REQ_VERSION: expected version 1. Check the request template and retry." The new message supplies a stable category and an action without echoing a full untrusted payload. For an uncertain post-submission timeout, a different message must say to retry the same ID and payload because work may already have committed. Reusing the invalid-input wording would give the operator the wrong recovery model.

Runnable example

File: `python/examples/lesson_20_4.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.4 --build-dir build
```

```
old="ERROR: invalid"
new="REQ_VERSION: expected version 1. Check the request template and retry."
print("Before:",old)
print("After:",new)
print("Review: text-only status, actionable recovery, no secret or full payload in the message.")
```

Expected output from the supplied reference in the validated environment:

```
Before: ERROR: invalid
After: REQ_VERSION: expected version 1. Check the request template and retry.
Review: text-only status, actionable recovery, no secret or full payload in the message.
```

Practical checkpoint

Run an accessibility review and a blameless failure retrospective on the project. Pass: identify at least three actionable changes, assign acceptance evidence and document third-party attribution honestly.

Expanded contract: Evidence lab: usability/accessibility review, blameless incident report, attribution/dependency record and Gate D handoff. Do not mark human review complete because an automated example printed a proposed message.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

An optimization removes clear errors and silently retries forever. Is higher throughput sufficient justification?

Quiz — answer before opening the key

1. **Does public availability imply permission to redistribute?**
2. **Why include accessibility in acceptance criteria?**
3. **What should a blameless retrospective seek?**
4. **Why must UNKNOWN_OUTCOME have different recovery instructions from INVALID_REQUEST?**
5. **What makes a retrospective action testable?**

Assigned study

R29: input, text, captions and UI guidance; R01/R02: professional and ethical practice.

Expansion source map: R01, R02, R29. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Complete Gate D: a reviewed systems application with a design record, threat model, CI-style local test pipeline, dependency inventory and an incident/recovery exercise.

Assemble the earlier investigations into the local workload-processing application. Complete the Gate D specification rather than adding a public deployment.

1. Six measurable requirements, two reviewed decision records and test-to-requirement traceability.
2. A thread/resource ownership map, protocol/schema and explicit commit/cancel boundary.
3. A bounded threat model and malformed-input harness; residual risks and diagnostic limits.
4. A fresh build, versioned source archive and manifest; intentionally failed tests prevent release.
5. An operator/accessibility review, blameless recovery retrospective, dependency inventory and independent handoff.

Live review: Give a reviewer a fresh extraction. Reproduce one recovery case, explain one measured bottleneck and implement a small changed requirement without copying a solution.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.

Sources and reading directory

The original lesson IDs, titles, outcomes, primers, examples, lab briefs, reading assignments and three-question quizzes come from the supplied `curriculum.json`, edition 2026-09-28. They are retained in `source/original_volume_04.json` and the `source` fields of `source/expanded_curriculum.json`. The original file SHA-256 is `21ffde3c0e228c7729933e4e92010295c7a46001fd05c995090ad79dde773a25`.

The expanded explanations, implementation contracts, code, fifty additional exercises, two application questions per lesson, module reviews and ForgeQueue design are newly authored for Package 05. They are not quoted textbook chapters or completed student work. External books and specifications are linked, not bundled in full.

R-identifiers preserve the source program directory. P-identifiers identify primary documentation consulted for this expansion on September 28, 2026. A moving documentation page is not a pin for the installed tool version; the measured validation environment is recorded separately. R37 remains an inherited optional GPU comparison, not a prerequisite for running the CPU labs. P15 is unused.

The source Module 20 prerequisite literally reads “Volumes 01–04.” For sequencing within this volume, this expansion interprets that as the preceding foundations and Modules 16–19 before Module 20, rather than requiring Module 20 to be completed before itself. The source text is retained unchanged in the source data.

R01 — CS2023 knowledge areas

ACM / IEEE-CS / AAAI

Breadth checklist, not a claim of accreditation.

Open the primary/source resource

R02 — Computer Engineering Curricula 2016

ACM / IEEE Computer Society

Computer-engineering breadth and laboratory expectations; historical curriculum report.

Open the primary/source resource

R04 — Python tutorial and language documentation

Python Software Foundation

Reference companion; the tutorial itself assumes some programming familiarity.

Open the primary/source resource

R05 — Pro Git

Scott Chacon and Ben Straub

Version control and collaboration.

Open the primary/source resource

R11 — C++ Core Guidelines

C++ Core Guidelines editors

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

Open the primary/source resource

R12 — CMake tutorial

Kitware

Build systems; use documentation matching the installed release.

Open the primary/source resource

R15 — 6.004 Computation Structures

MIT OpenCourseWare

Logic, processor design, memory and systems.

Open the primary/source resource

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau

Virtualization, concurrency and persistence; book by its authors.

Open the primary/source resource

R17 — RFC 9293: Transmission Control Protocol

IETF / RFC Editor

Transport semantics; reference reading after networking foundations.

Open the primary/source resource

R18 — PostgreSQL tutorial

PostgreSQL Global Development Group

Relational data, transactions and SQL; pin local server and client versions.

Open the primary/source resource

R29 — Xbox Accessibility Guidelines

Microsoft

Accessibility design and testing; not a legal-compliance certificate.

Open the primary/source resource

R35 — RISC-V ISA specifications

RISC-V International

Instruction-set reference; record the exact ISA version and implemented extensions.

Open the primary/source resource

R37 — CUDA Programming Guide

NVIDIA

GPU execution and memory; CUDA is an optional vendor-specific lab route.

Open the primary/source resource

P01 — RISC-V ELF psABI: integer register and calling conventions

RISC-V non-ISA specifications project

Use the ILP32 integer convention for the teaching RV32 routine; no floating-point ABI claim.

Open the primary/source resource

P02 — Developer options: optimization reports

GNU Compiler Collection

Match options and interpretation to the installed compiler.

Open the primary/source resource

P03 — Clang user manual: optimization remarks

LLVM Project

Compiler diagnostic interface, not a performance guarantee.

Open the primary/source resource

P04 — AddressSanitizer

LLVM Project

Instrumented checks; platform support and observed coverage have limits.

Open the primary/source resource

P05 — ThreadSanitizer

LLVM Project

Data-race detection; separate from ASan and not a proof of liveness.

Open the primary/source resource

P06 — os: replacement and synchronization interfaces

Python Software Foundation

File and directory durability require a platform-specific failure model.

Open the primary/source resource

P07 — mmap

Python Software Foundation

Read-only mapping and platform-specific behavior.

Open the primary/source resource

P08 — subprocess

Python Software Foundation

Argument lists, checked results and timeout behavior for a trusted program.

Open the primary/source resource

P09 — socket

Python Software Foundation

Partial reads, socket waits and connection behavior.

Open the primary/source resource

P10 — sqlite3

Python Software Foundation

Connection and transaction handling. Tested runtime is recorded separately.

Open the primary/source resource

P11 — Foreign key support

SQLite project

Explicit per-connection enforcement in the lab.

Open the primary/source resource

P12 — Transactions

SQLite project

BEGIN IMMEDIATE, writer behavior and explicit transactions.

Open the primary/source resource

P13 — CTest command-line reference

Kitware

Use documentation matching the installed CMake/CTest version.

Open the primary/source resource

P14 — concurrent.futures

Python Software Foundation

Future completion, executor shutdown and cancellation contracts.

Open the primary/source resource

P16 — RV32I base integer ISA, version 2.1; library snapshot v20260120

RISC-V International

Pinned specification entry for instruction semantics; no full-ISA emulator is supplied.

Open the primary/source resource